

DOCUMENTACIÓN DEL PROYECTO

SISTEMA DE ACTUALIZACIÓN AUTOMATIZADA DE LA MACRO INVENTARIO GENERAL

Nombre del proyecto: Sistema de Actualización Automatizada de Macro Inventario General (MACRO_INV_GENERAL)

Fecha de creación: Octubre 2025

Responsable de desarrollo: Departamento de Tecnología e Innovación

Responsable de documentación: Jefferson Pérez Bedoya

Fecha de última actualización: 22 de octubre de 2025

Versión: 1.0

Descripción breve:

Automatización del proceso de actualización del inventario general de FERTRAC mediante scripts en Python que consolidan información de múltiples archivos Excel descargados del ERP, calculan existencias netas, actualizan costos promedios y clasifican productos según reglas de negocio establecidas. El proceso se ejecuta mediante archivos .bat que permiten la ejecución con un solo clic.

1. DESCRIPCIÓN GENERAL DEL DESARROLLO

Este desarrollo permite actualizar automáticamente la macro inventario general consolidando información de diferentes fuentes del sistema ERP sin necesidad de realizar el proceso manualmente.

El sistema está diseñado para operar en dos momentos del día:

- **Proceso mediodía:** Actualización principal del inventario general
- **Proceso tarde:** Actualización complementaria con datos adicionales del día

¿Qué hace el código?

El proceso automatizado:

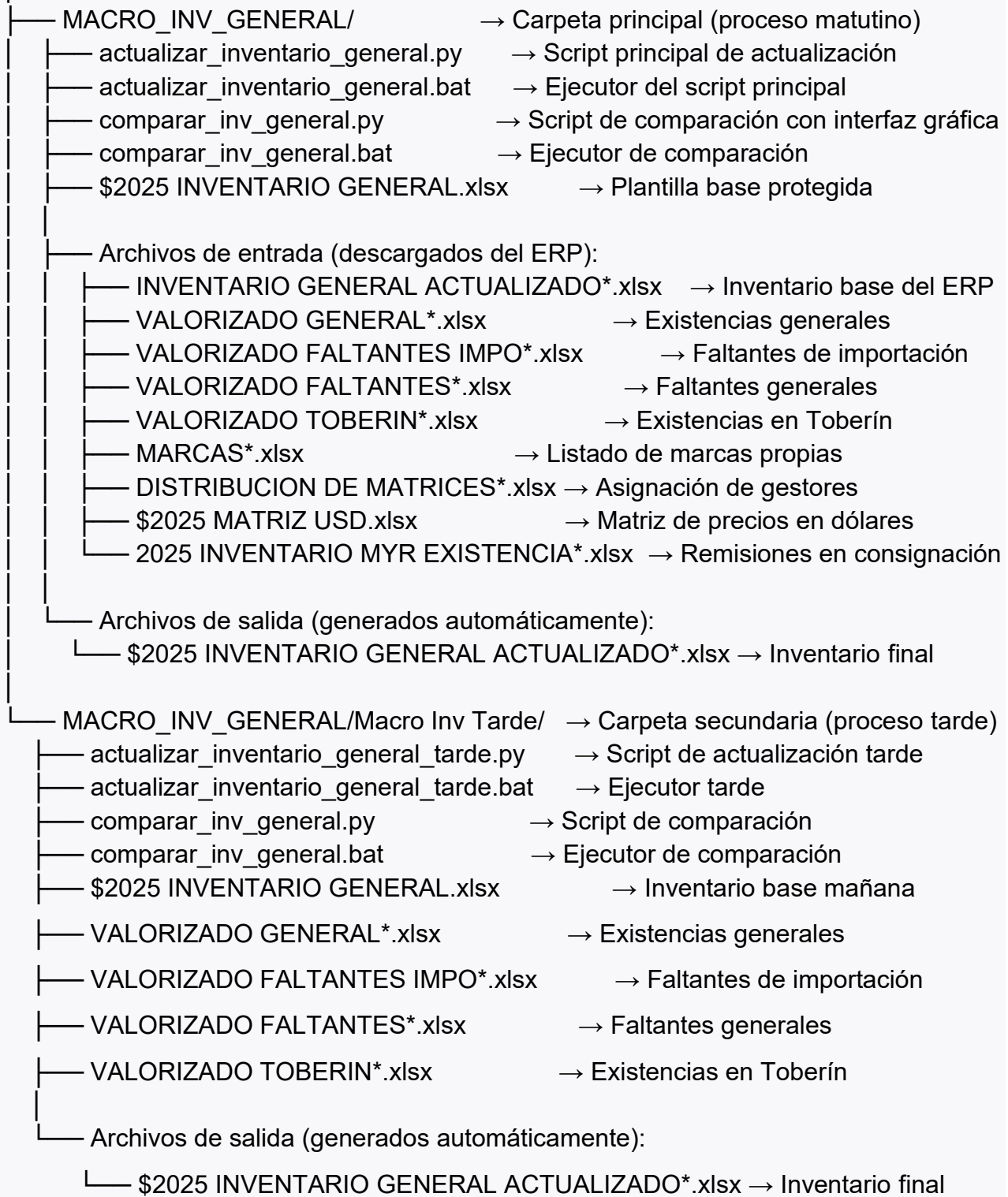
1. Lee múltiples archivos Excel con contraseña del sistema ERP
2. Normaliza referencias de productos para evitar duplicados
3. Calcula existencias netas aplicando la fórmula: **VALORIZADO GENERAL - FALTANTES IMPO - FALTANTES - TOBERÍN**
4. Actualiza costos promedios desde archivos valorizados
5. Asigna clasificaciones (marcas, líneas, sub-líneas, líderes) según archivos maestros
6. Aplica reglas especiales para marcas propias
7. Genera un archivo Excel actualizado con protección por contraseña
8. Permite comparar inventarios manuales vs automatizados para validación

¿Por qué se desarrolló?

Para eliminar el proceso manual que tomaba varias horas diarias, reducir errores humanos en la consolidación de datos y garantizar que el inventario esté siempre actualizado con información del ERP.

2. ESTRUCTURA DEL PROYECTO

Escritorio/Proyectos/



Descripción de cada elemento:

Archivo/Carpeta	Propósito
actualizar_inventario_general.py	Script principal que consolida todos los archivos y genera el inventario actualizado
actualizar_inventario_general.bat	Archivo de ejecución que lanza el script Python con un doble clic
comparar_inv_general.py	Herramienta de validación que compara inventarios y genera reporte de diferencias
comparar_inv_general.bat	Ejecutor de la herramienta de comparación
\$2025 INVENTARIO GENERAL.xlsx	Plantilla maestra con estructura, formato y fórmulas base
Archivos VALORIZADO*	Reportes del ERP con existencias y costos por ubicación
MARCAS.xlsx	Listado de marcas que pertenecen a la empresa
DISTRIBUCION DE MATRICES.xlsx	Asignación de líderes y clasificaciones por línea de producto
\$2025 MATRIZ USD.xlsx	Referencias y descripciones para lista de precios

3. DESCRIPCIÓN TÉCNICA DEL PROCESO

Flujo Principal - actualizar_inventario_general.py

Paso 1: El usuario ejecuta el archivo actualizar_inventario_general.bat

Paso 2: El archivo .bat busca Python en el sistema y ejecuta el script actualizar_inventario_general.py

Paso 3: El script realiza las siguientes operaciones en secuencia:

1. Carga de archivos fuente

- Busca archivos por prefijo en la carpeta del proyecto
- Intenta abrir con openpyxl (motor de lectura Excel para Python)
- Si está cifrado, prueba contraseñas configuradas
- Si falla, usa Excel COM (interfaz de Windows) para convertir el archivo
- Lee los datos con el encabezado correcto según tipo de archivo

2. Normalización de referencias

- Aplica función to_num_str() que convierte referencias como "95.276" → "95276"
- Elimina decimales innecesarios y separadores de miles
- Mantiene referencias alfanuméricas sin cambios (ej: "FP-1234")

3. Cálculo de existencias consolidadas

- Lee cantidad del VALORIZADO GENERAL
- Resta cantidades de: FALTANTES IMPO, FALTANTES, TOBERÍN
- Fórmula: $EXIST_FINAL = VAL_GENERAL - VAL_IMPO - VAL_FALT - VAL_TOBERIN$
- Genera diccionario {referencia: existencia_calculada}

4. Extracción de costos promedios

- Lee columna "COSTO PROMEDIO" de archivos valorizados
- Crea diccionario {referencia: costo_promedio}
- Convierte valores a numérico, asigna 0 si no existe

5. Apertura del archivo plantilla con Excel COM

- Abre \$2025 INVENTARIO GENERAL.xlsx con contraseña
- Si está cifrado, lo descripta a archivo temporal
- Desactiva cálculo automático para mayor velocidad
- Localiza hojas: INVENTARIO, INVENTARIO COPIA, INV LISTA PRECIOS

6. Actualización de hoja INVENTARIO COPIA

- Limpia columnas especificadas en COLS_A_LIMPIAR
- Actualiza encabezado de EXISTENCIA con fecha del día (formato: "EXISTENCIA OCT 22")
- Lee referencias de la columna REFERENCIA (fila por fila)
- Para cada referencia, busca en diccionarios y escribe:
 - Existencia calculada
 - Costo promedio
 - Nombre desde inventario ERP
 - Marca, línea, sub-línea desde archivos maestros
- Copia datos desde hoja INVENTARIO original (MARCA copia, INV BODEGA GERENCIA, etc.)

7. Aplicación de reglas de marcas propias

- Filtra registros donde LINEA COPIA esté vacía o sea "INDETERMINADO"
- Y donde MARCA SISTEMA esté en el listado de marcas propias
- Elimina referencias tipo "0041R" (referencias obsoletas)

- Actualiza campos según archivo DISTRIBUCION:
 - LINEA COPIA = MARCA SISTEMA
 - SUB-LINEA COPIA = MARCA SISTEMA
 - LIDER LINEA = gestor asignado en distribución
 - CLASIFICACION = clasificación asignada
 - INV BODEGA GERENCIA = valor de remisión en consignación

8. Actualización de hoja INV LISTA PRECIOS

- Limpia columnas: REFERENCIA LISTA, NOMBRE LISTA
- Busca cada referencia en Matriz USD
- Escribe REFERENCIA LISTA DE PRECIOS y DESCRIPCION LISTA PRECIOS

9. Guardado del archivo final

- Reactiva cálculo automático de Excel
- Guarda el archivo con timestamp en el nombre: \$2025 INVENTARIO GENERAL ACTUALIZADO YYYYMMDD_HHMM.xlsx
- Aplica contraseña de protección si está configurado
- Elimina archivos temporales
- Cierra Excel COM

10. Registro de log

- Imprime en consola cada paso realizado con timestamp
- Indica cantidad de registros procesados
- Muestra estadísticas (referencias actualizadas, errores, etc.)

Flujo de Comparación - comparar_inv_general.py

Paso 1: Usuario ejecuta comparar_inv_general.bat

Paso 2: Se abre interfaz gráfica (Tkinter) con 2 diálogos:

- Seleccionar archivo INVENTARIO MANUAL
- Seleccionar archivo INVENTARIO AUTOMATIZADO (generado por el script)

Paso 3: El script realiza:

1. Lee ambos archivos (maneja contraseñas, cifrado, formatos .xls/.xlsx/.csv)
2. Detecta automáticamente la hoja correcta (busca "INVENTARIO", "INVENTARIO GENERAL", etc.)
3. Localiza fila de encabezado (busca columna "REFERENCIA" en las primeras 10 filas)
4. Normaliza referencias con to_num_str()
5. Identifica columnas comunes entre ambos archivos
6. **Búsqueda especial:** Detecta columnas que empiezan con "EXISTENCIA" aunque tengan nombres diferentes
7. Compara valor por valor cada columna común
8. Normaliza valores para comparación:
 - Convierte errores de Excel (#N/D, #DIV/0!) a cadena vacía
 - Elimina espacios
 - Compara numéricamente con tolerancia de 0.000001
9. Genera DataFrame con diferencias encontradas: REFERENCIA, COLUMNA, VALOR_MANUAL, VALOR_AUTO
10. Escribe archivo Excel: DIFERENCIAS INVENTARIO YYYYMMDD_HHMM.xlsx
11. Aplica formato: auto-ajuste de columnas, filtros automáticos

Paso 4: Muestra en consola la ruta del archivo generado

4. DOCUMENTACIÓN DE CADA SCRIPT

4.1 actualizar_inventario_general.py

Descripción general:

Script principal que consolida información de múltiples archivos Excel del ERP para generar un inventario actualizado con existencias, costos y clasificaciones.

Librerías utilizadas:

- pandas: Lectura y manipulación de datos Excel/CSV
- openpyxl: Motor de lectura/escritura Excel (formato .xlsx)
- msoffcrypto: Descriptación de archivos Excel protegidos
- win32com.client: Interfaz COM para controlar Excel de Windows
- unidecode: Normalización de texto (elimina tildes)
- pathlib: Manejo de rutas de archivos
- datetime: Gestión de fechas y timestamps
- tempfile: Creación de archivos temporales
- warnings: Supresión de advertencias

Variables de configuración principales:

```
BASE_PATH = Path(__file__).resolve().parent # Carpeta donde está el script
PASS_INV = "Compras2025" # Contraseña del inventario
PASSWORDS_TRY = ["Compras2025", "Compras2026"] # Contraseñas a probar

# Nombres de archivos a buscar
PFX_INV_ACTUALIZADO = "INVENTARIO GENERAL ACTUALIZADO"
PFX_VAL_GENERAL = "VALORIZADO GENERAL"
PFX_VAL_FALT_IMPO = "VALORIZADO FALTANTES IMPO"
PFX_VAL_FALT = "VALORIZADO FALTANTES"
PFX_VAL_TOBERIN = "VALORIZADO TOBERIN"
PFX_MARCAS = "MARCAS"
```

```
PFX_DISTRIBUCION = "DISTRIBUCION DE MATRICES"
PFX_MAYOR_EXISTENCIA = "2025 INVENTARIO MYR EXISTENCIA"
PFX_MATRIZ_USD = "$2025 MATRIZ USD"

# Archivo plantilla
FN_INV_PLANTILLA = "$2025 INVENTARIO GENERAL.xlsx"

# Nombres de hojas
SHEET_INV_ORIG = "INVENTARIO"
SHEET_INV_COPIA = "INVENTARIO COPIA"
SHEET_INV_LISTA = "INV LISTA PRECIOS"

# Filas de encabezado (1-based)
HEADER_ROW_INV = 2      # Fila de encabezado en hoja INVENTARIO
HEADER_ROW_VAL = 9      # Fila de encabezado en archivos VALORIZADO
HEADER_ROW_MATRIZ = 1   # Fila de encabezado en Matriz USD
```

Funciones principales:

to_num_str(x)

Propósito: Normaliza referencias para evitar duplicados causados por formatos numéricos diferentes.

Parámetros:

- x: Valor a normalizar (puede ser string, int, float, None)

Retorna: String normalizado sin decimales ni separadores

Lógica:

- Si es vacío/None → retorna ""
- Si contiene letras → lo mantiene tal cual (ej: "FP-1234")
- Si es numérico → elimina punto decimal si es entero (95.0 → "95")

- Maneja separadores de miles (95.276 → "95276")

Ejemplo:

```
to_num_str("95.276") → "95276"  
to_num_str(95.0) → "95"  
to_num_str("FP-1234") → "FP-1234"  
to_num_str(None) → ""
```

find_by_prefix(base_dir: Path, prefix: str, exts=(".xlsx", ".xlsm", ".xls", ".csv"))

Propósito: Busca archivos en una carpeta por prefijo normalizado (ignora tildes, mayúsculas, caracteres especiales).

Parámetros:

- `base_dir`: Carpeta donde buscar
- `prefix`: Texto inicial del nombre de archivo
- `exts`: Extensiones permitidas

Retorna: Path del archivo más reciente que coincide

Lógica:

1. Normaliza el prefijo con `_norm()` (minúsculas, sin tildes)
2. Recorre archivos de la carpeta
3. Normaliza nombre de cada archivo
4. Verifica si empieza con el prefijo o contiene todas sus palabras
5. Ordena candidatos por fecha de modificación (más reciente primero)
6. Retorna el primero

Manejo de errores: Lanza `FileNotFoundError` si no encuentra ningún archivo

decrypt_to_stream(xlsx_path: Path, password: str)

Propósito: Desencrpta archivos Excel protegidos con contraseña.

Parámetros:

- `xlsx_path`: Ruta del archivo cifrado
- `password`: Contraseña de descryptación

Retorna: `io.BytesIO` objeto en memoria con el archivo descryptado

Requiere: Librería `msoffcrypto`

Proceso:

1. Abre archivo en modo binario
2. Crea objeto `msoffcrypto.OfficeFile`
3. Carga la contraseña
4. Descrypta a buffer en memoria (`BytesIO`)
5. Resetea posición del buffer a inicio

`com_convert_to_xlsx(path: Path, passwords: list)`

Propósito: Convierte archivos `.xls` o archivos problemáticos a `.xlsx` usando Excel COM.

Parámetros:

- `path`: Archivo a convertir
- `passwords`: Lista de contraseñas a probar si está cifrado

Retorna: Path del archivo temporal `.xlsx` generado

Requiere: Excel instalado en Windows + librería `win32com`

Proceso:

1. Inicia instancia invisible de Excel
2. Desactiva alertas y actualizaciones automáticas
3. Detecta si el archivo está cifrado
4. Intenta abrir con cada contraseña de la lista

5. Guarda como .xlsx en carpeta temporal
6. Cierra Excel y retorna ruta del temporal

cargar_inventario_actualizado(base_dir: Path)

Propósito: Carga el archivo principal de inventario del ERP o plantilla de respaldo.

Parámetros:

- `base_dir`: Carpeta del proyecto

Retorna: DataFrame con columnas:

- `__REFERENCIA__`: Referencia normalizada
- `__NOMBRE__`: Nombre del producto
- `__MARCA_SYS__`: Marca del sistema
- `__LINEA_SYS__`: Línea del sistema
- `__SUBLINEA_SYS__`: Sub-línea del sistema
- `__COSTO__`: Costo del producto

Lógica:

1. Busca archivo "INVENTARIO GENERAL ACTUALIZADO" (preferido)
2. Si no existe, usa plantilla "\$2025 INVENTARIO GENERAL"
3. Abre con manejo de contraseñas
4. Detecta hoja correcta ("Sheet 1" en ERP, "INVENTARIO" en plantilla)
5. Busca columna de REFERENCIA de forma flexible
6. Filtra filas vacías
7. Normaliza referencias con `to_num_str()`
8. Identifica y renombra columnas relevantes

cargar_valorizado(base_dir: Path, prefix: str)

Propósito: Lee archivos VALORIZADO del ERP con cantidades y costos.

Parámetros:

- `base_dir`: Carpeta del proyecto
- `prefix`: Prefijo del archivo ("VALORIZADO GENERAL", etc.)

Retorna: DataFrame con:

- `__REF_INT__`: Referencia normalizada
- `__CANT__`: Cantidad (numérico)
- `__COSTO__`: Costo promedio (numérico, 0 si no existe)

Lógica:

1. Busca archivo por prefijo
2. Lee todo el archivo sin encabezado
3. Localiza fila de encabezado (configurada en `HEADER_ROW_VAL = 9`)
4. Re-lee con encabezado correcto
5. Busca columnas: "REFERENCIA INTERNA", "CANTIDAD", "COSTO PROMEDIO"
6. Normaliza referencias
7. Convierte cantidad y costo a numérico
8. Retorna datos limpios

cargar_matriz_usd(base_dir: Path)

Propósito: Carga la matriz de precios con referencias para lista de precios.

Parámetros:

- `base_dir`: Carpeta del proyecto

Retorna: DataFrame con:

- `__REF_MATRIZ__`: Referencia de inventario

- `__DESC_LISTA__`: Descripción para lista de precios
- `__REF_LISTA_PRECIOS__`: Referencia alternativa para lista

Lógica:

1. Busca archivo "\$2025 MATRIZ USD"
2. Localiza hoja "2025"
3. Detecta automáticamente fila de encabezado (busca "REFERENCIA" y "DESCRIPCION")
4. Identifica columnas:
 - "REFERENCIA INVENTARIO FERTRAC"
 - "DESCRIPCION LISTA PRECIOS"
 - "REFERENCIA LISTA DE PRECIOS"
5. Elimina duplicados (conserva primer registro)

cargar_marcas(base_dir: Path)

Propósito: Carga el listado de marcas que pertenecen a la empresa.

Parámetros:

- `base_dir`: Carpeta del proyecto

Retorna: set con marcas en mayúsculas

Lógica:

1. Busca archivo "MARCAS"
2. Lee las primeras 3 columnas
3. Extrae valores únicos que contengan letras
4. Convierte a mayúsculas
5. Filtra valores inválidos ("NONE", "NAN", "MARCA")

cargar_distribucion(base_dir: Path)

Propósito: Carga asignación de gestores y clasificaciones por línea de producto.

Parámetros:

- `base_dir`: Carpeta del proyecto

Retorna: Diccionario con estructura:

```
{  
  'gestor': {'LINEA1': 'Gestor A', 'LINEA2': 'Gestor B'},  
  'clasificacion': {'LINEA1': 'Categoría X', 'LINEA2': 'Categoría Y'}  
}
```

Lógica:

1. Busca archivo "DISTRIBUCION DE MATRICES"
2. Detecta fila de encabezado (busca "LINEA" y "GESTOR")
3. Lee columnas: LINEA, GESTOR, CLASIFICACION
4. Limpia nombres de línea (elimina paréntesis)
5. Crea diccionarios de mapeo

cargar_mayor_existencia(base_dir: Path)

Propósito: Carga valores de remisiones en consignación.

Parámetros:

- `base_dir`: Carpeta del proyecto

Retorna: DataFrame con:

- `__REF_MAYOR__`: Referencia
- `__REM_CONSIG__`: Remisión en consignación (numérico)

Lógica:

1. Busca archivo "2025 INVENTARIO MYR EXISTENCIA"

2. Localiza hoja "COSTOS INV FINAL"
3. Detecta encabezado
4. Busca columnas: "REFERENCIA FERTRAC", "REM EN CONSIG"
5. Convierte remisión a numérico

excel_open(path: Path, password: str)

Propósito: Abre archivo Excel con Excel COM, manejando cifrado y contraseñas.

Parámetros:

- path: Archivo a abrir
- password: Contraseña (opcional)

Retorna: Tupla (excel_app, workbook, info_dict)

Proceso:

1. Inicia Excel COM invisible
2. Detecta si está cifrado
3. Si está cifrado, descripta a temporal
4. Abre el archivo (original o temporal)
5. Desactiva cálculo automático
6. Retorna aplicación, libro y diccionario con información

aplicar_reglas_marcas_propias(...)

Propósito: Aplica lógica de negocio para productos de marcas propias.

Parámetros:

- ws_inv_copia: Hoja Excel de destino
- start_data_row, last_row: Rango de datos
- ref_col_idx: Índice de columna de referencia

- `hdrn_copia`: Diccionario de encabezados normalizados
- `marcas_propias`: Set de marcas
- `distribucion`: Diccionario de asignaciones

Lógica:

1. **Fase 1 - Eliminación:** Identifica y elimina referencias tipo "0041R" (obsoletas)
2. **Fase 2 - Filtrado:** Selecciona registros donde:
 - LINEA COPIA esté vacía o sea "INDETERMINADO"
 - MARCA SISTEMA esté en marcas propias
3. **Fase 3 - Actualización:** Para registros filtrados:
 - LINEA COPIA = MARCA SISTEMA
 - SUB-LINEA COPIA = MARCA SISTEMA
 - LIDER LINEA = gestor de distribución
 - CLASIFICACION = clasificación de distribución
 - INV BODEGA GERENCIA = remisión en consignación

`main()`

Propósito: Función principal que orquesta todo el proceso.

Flujo completo:

1. Validar que Excel COM esté disponible
2. Cargar todos los archivos fuente
3. Crear mapas de cantidades desde valorizados
4. Calcular existencias: GENERAL - IMPO - FALT - TOBERIN
5. Crear diccionarios de existencias y costos
6. Abrir plantilla con Excel COM
7. Actualizar hoja INVENTARIO COPIA:
 - Limpiar columnas
 - Actualizar encabezado EXISTENCIA con fecha

- Escribir existencias, costos, nombres
 - Copiar datos desde hoja original
8. Aplicar reglas de marcas propias
 9. Actualizar hoja INV LISTA PRECIOS
 10. Guardar archivo con timestamp
 11. Aplicar contraseña si está configurado
 12. Limpiar temporales y cerrar Excel
 13. Mostrar resumen en consola

4.2 comparar_inv_general.py

Descripción general:

Herramienta de validación que compara dos archivos de inventario y genera un reporte Excel con las diferencias encontradas.

Librerías utilizadas:

- pandas: Manipulación de datos
- openpyxl: Lectura/escritura Excel
- tkinter: Interfaz gráfica para selección de archivos
- msoffcrypto: Manejo de archivos cifrados
- win32com: Conversión de archivos problemáticos
- pathlib: Gestión de rutas

Variables de configuración:

```
SHEET_CANDIDATES = ["INVENTARIO", "INVENTARIO GENERAL", "INV", "Sheet1",  
"Hoja1"]  
SEARCH_HEADER_MAXROW = 10 # Máximo de filas a buscar el encabezado  
COLUMNAS_A_COMPARAR = None # None = todas las columnas comunes  
CAMPOS_IGNORAR = {"Unnamed: 0"}  
PASSWORDS_TRY = ["Compras2025", "Compras2026"]
```

```
ALLOW_COM_CONVERSION = True  
VERBOSE = True # Mostrar logs detallados
```

Funciones principales:

`_norm(s: str)`

Propósito: Normaliza texto para comparaciones insensibles a mayúsculas/tildes.

Parámetros:

- `s`: String a normalizar

Retorna: String en minúsculas, sin tildes ni caracteres especiales

Ejemplo:

```
_norm("Línea/Descripción") → "linea descripcion"  
_norm("REFERENCIA INTERNA") → "referencia interna"
```

`find_header_row(df: DataFrame)`

Propósito: Detecta automáticamente qué fila contiene el encabezado.

Parámetros:

- `df`: DataFrame sin encabezado

Retorna: Índice de la fila de encabezado (int) o None

Lógica:

- Recorre las primeras 10 filas
- Busca valores normalizados: "referencia", "codigo", "sku", "ref"
- Retorna índice de la primera coincidencia

`pick_sheet(xls: ExcelFile)`

Propósito: Selecciona la mejor hoja de un libro Excel.

Parámetros:

- xls: Objeto ExcelFile de pandas

Retorna: Nombre de la hoja seleccionada

Lógica:

1. Crea mapa normalizado de nombres de hojas
2. Busca coincidencia exacta con candidatos
3. Si no encuentra, busca contenido parcial
4. Si falla todo, retorna la primera hoja

read_inventory(path: Path)

Propósito: Lee un archivo de inventario manejando múltiples formatos y problemas.

Parámetros:

- path: Ruta del archivo

Retorna: DataFrame limpio con columna `__REFERENCIA__` normalizada

Proceso completo:

1. Abre archivo (maneja cifrado, .xls, CSV)
2. Selecciona hoja correcta
3. Detecta fila de encabezado
4. Re-lee con encabezado correcto
5. Elimina columnas "Unnamed"
6. Hace únicos los nombres de columnas (Col, Col__2, Col__3)
7. Busca columna de REFERENCIA
8. Filtra filas vacías

9. Normaliza referencias con `to_num_str()`

10. Elimina duplicados (conserva último)

`norm_for_compare(v)`

Propósito: Normaliza valores para comparación precisa.

Parámetros:

- `v`: Valor a normalizar (puede ser cualquier tipo)

Retorna: Valor normalizado para comparación

Lógica:

- Si es Serie (columna duplicada) → toma primer valor
- Si es None/NaN → retorna ""
- Si es error de Excel (#N/D, #DIV/0!) → retorna ""
- Si es string → elimina espacios
- Mantiene otros tipos tal cual

`comparar_en_una_hoja(...)`

Propósito: Función principal que compara dos inventarios.

Parámetros:

- `path_manual`: Archivo inventario manual
- `path_auto`: Archivo inventario automatizado
- `out_path`: Ruta del archivo de salida
- `columnas_objetivo`: Lista de columnas a comparar (None = todas)

Proceso:

1. Lee ambos archivos con `read_inventory()`
2. Identifica columnas comunes

3. **Detección especial:** Busca columnas "EXISTENCIA*" aunque tengan nombres diferentes
4. Crea índices por REFERENCIA (sin duplicados)
5. Para cada referencia común:
 - Compara cada columna
 - Normaliza valores con `norm_for_compare()`
 - Si son diferentes:
 - Intenta comparación numérica con tolerancia
 - Si siguen siendo diferentes, registra la diferencia
6. Genera DataFrame con: REFERENCIA, COLUMNA, VALOR_MANUAL, VALOR_AUTO
7. Escribe Excel con formato:
 - Auto-ajuste de columnas
 - Filtros automáticos
8. Muestra estadísticas en consola

main()

Propósito: Lanza interfaz gráfica y coordina el proceso.

Flujo:

1. Crea ventana Tkinter invisible (root)
2. Configura como "topmost" (siempre al frente)
3. Muestra diálogo para seleccionar archivo MANUAL
4. Muestra diálogo para seleccionar archivo AUTO
5. Define nombre de salida: DIFERENCIAS INVENTARIO YYYYMMDD_HHMM.xlsx
6. Ejecuta comparación
7. Muestra ruta del archivo generado

8. Maneja errores y muestra en consola

5. DOCUMENTACIÓN DE ARCHIVOS .BAT

5.1 actualizar_inventario_general.bat

Contenido:

```
@echo off
cd /d "%~dp0"
python actualizar_inventario_general.py
pause
```

Explicación línea por línea:

Línea	Propósito
@echo off	Oculto comandos en pantalla, muestra solo resultados
cd /d "%~dp0"	Cambia directorio actual a la carpeta donde está el .bat (%~dp0 = ruta del script)
python actualizar_inventario_general.py	Ejecuta el script Python en el mismo directorio
pause	Espera que el usuario presione una tecla antes de cerrar la ventana

¿Cómo funciona?

Al hacer doble clic en el archivo .bat:

1. Se abre una ventana de consola (CMD)
2. Cambia a la carpeta del proyecto

3. Busca Python en el PATH del sistema
4. Ejecuta el script de actualización
5. Muestra todos los logs del proceso
6. Al terminar, espera que presiones Enter para cerrar

5.2 comparar_inv_general.bat

Contenido:

```
@echo off
chcp 65001 >nul
title Comparar Inventarios - Lanzador
setlocal

REM Ruta del script (mismo folder que este .bat)
set "SCRIPT=%~dp0comparar_inv_general.py"

if not exist "%SCRIPT%" (
    echo No se encontro el script: "%SCRIPT%"
    echo Pon este .bat en la misma carpeta que comparar_inv_general.py
    pause
    exit /b 1
)

REM Si hay venv local, usarlo primero
set "VENV_PY=%~dp0venv\Scripts\python.exe"
if exist "%VENV_PY%" (
    echo Usando entorno virtual: "%VENV_PY%"
    "%VENV_PY%" "%SCRIPT%"
    goto :postrun
)

REM Probar con el Python Launcher (py)
where py >nul 2>&1
```

```

if %errorlevel%==0 (
    echo Usando py launcher...
    py -3 "%SCRIPT%"
    goto :postrun
)

REM Probar con python en PATH
where python >nul 2>&1
if %errorlevel%==0 (
    echo Usando python del sistema...
    python "%SCRIPT%"
    goto :postrun
)

echo No se encontro Python en el sistema.
echo Instala Python desde https://www.python.org/downloads/ o usa un entorno virtual
"venv".
pause
exit /b 1

:postrun
set "RC=%ERRORLEVEL%"
echo.
if "%RC%"=="0" (
    echo Proceso completado correctamente.
) else (
    echo El script termino con codigo de salida %RC%.
)
echo.
pause
endlocal

```

Explicación de secciones:

1. Configuración inicial:

```
@echo off          ← Oculta comandos
chcp 65001 >nul     ← Configura UTF-8 para caracteres especiales
title Comparar Inventarios ← Título de la ventana
setlocal           ← Crea ámbito local para variables
```

2. Validación de existencia del script:

```
set "SCRIPT=%~dp0comparar_inv_general.py" ← Define ruta del script
if not exist "%SCRIPT%" (                ← Si no existe
    echo No se encontro el script        ← Muestra error
    pause                                ← Espera
    exit /b 1                            ← Sale con código de error
)
```

3. Búsqueda de Python (en orden de preferencia):

a) Entorno virtual local (venv):

```
set "VENV_PY=%~dp0venv\Scripts\python.exe"
if exist "%VENV_PY%" (
    "%VENV_PY%" "%SCRIPT%"
    goto :postrun
)
```

b) Python Launcher (py):

```
where py >nul 2>&1    ← Busca comando 'py'
if %errorlevel%==0 ( ← Si existe
    py -3 "%SCRIPT%" ← Ejecuta con Python 3
    goto :postrun
)
```

c) Python en PATH:

```
where python >nul 2>&1
if %errorlevel%==0 (
    python "%SCRIPT%"
    goto :postrun
)
```

4. Manejo de resultados:

```
:postrun
set "RC=%ERRORLEVEL%"      ← Guarda código de salida
if "%RC%"=="0" (          ← Si es 0 (éxito)
    echo Proceso completado correctamente.
) else (
    echo El script termino con codigo de salida %RC%.
)
pause
```

Diferencia con el .bat simple:

- Más robusto: prueba múltiples formas de encontrar Python
- Soporta entornos virtuales
- Mejor manejo de errores
- Mensajes más informativos

6. REQUISITOS TÉCNICOS

Software requerido:

Sistema operativo:

- Windows 10 o superior (64-bit)

- Requiere Excel instalado (necesario para la funcionalidad COM)

Python:

- Versión: 3.8 o superior (recomendado 3.11+)
- Instalación desde: <https://www.python.org/downloads/>
- **Importante:** Durante la instalación, marcar "Add Python to PATH"

Microsoft Excel:

- Excel 2016 o superior
- Parte de Microsoft Office 365 o versión standalone
- Necesario para funcionalidad COM (win32com)

Librerías Python requeridas:

Instalar con el comando:

```
pip install pandas openpyxl msoffcrypto-tool pywin32 unidecode
```

Desglose de librerías:

Librería	Versión mínima	Propósito
pandas	1.3.0+	Manipulación de datos Excel/CSV
openpyxl	3.0.0+	Lectura/escritura de archivos .xlsx
msoffcrypto-tool	4.10.0+	Desencriptación de archivos protegidos
pywin32	300+	Interfaz COM para controlar Excel
unidecode	1.2.0+	Normalización de texto (eliminar tildes)
numpy	1.20.0+	(Dependencia de pandas) Operaciones numéricas

Comando completo de instalación:

```
pip install pandas==2.0.3 openpyxl==3.1.2 msoffcrypto-tool==5.0.1 pywin32==306  
unidecode==1.3.6
```

Estructura de archivos de entrada:

Archivo plantilla (\$2025 INVENTARIO GENERAL.xlsx):

- Formato: .xlsx
- Protección: Cifrado con contraseña "Compras2025"
- Hojas requeridas:
 - "INVENTARIO" (fila de encabezado: 2)
 - "INVENTARIO COPIA" (fila de encabezado: 2)
 - "INV LISTA PRECIOS" (fila de encabezado: 1)

Archivos del ERP (descargados):

1. INVENTARIO GENERAL ACTUALIZADO:

- Formato: .xlsx o .xlsm
- Hoja: "Sheet 1"
- Encabezado: fila 1
- Columnas mínimas: "Referencia", "Nombre", "Marca/ Nombre a mostrar", "Costo"

2. VALORIZADO GENERAL:

- Formato: .xlsx
- Hoja: primera hoja
- Encabezado: fila 9
- Columnas requeridas: "Referencia interna", "Cantidad", "Costo promedio"

3. VALORIZADO FALTANTES IMPO:

- Igual estructura que VALORIZADO GENERAL

4. VALORIZADO FALTANTES:

- Igual estructura que VALORIZADO GENERAL

5. VALORIZADO TOBERIN:

- Igual estructura que VALORIZADO GENERAL

6. MARCAS:

- Formato: .xlsx
- Contenido: Listado de marcas propias (sin formato específico, solo valores)

7. DISTRIBUCION DE MATRICES:

- Formato: .xlsx
- Columnas: "LINEA", "GESTOR", "CLASIFICACION"
- Puede tener fila de encabezado variable (auto-detectada)

8. \$2025 MATRIZ USD:

- Formato: .xlsx (cifrado)
- Hoja: "2025"
- Columnas: "REFERENCIA INVENTARIO FERTRAC", "DESCRIPCION LISTA PRECIOS", "REFERENCIA LISTA DE PRECIOS"

9. 2025 INVENTARIO MYR EXISTENCIA:

- Formato: .xlsx
- Hoja: "COSTOS INV FINAL"
- Columnas: "REFERENCIA FERTRAC", "REM EN CONSIG" (columna AI)

Permisos y accesos necesarios:

- **Permisos de lectura:** En la carpeta del proyecto y subcarpetas
- **Permisos de escritura:** Para generar archivos de salida
- **Permisos de ejecución:** Para ejecutar archivos .bat y Python
- **Contraseñas conocidas:** "Compras2025", "Compras2026" (ya configuradas en el código)

Especificaciones de hardware:

Componente	Mínimo	Recomendado
Procesador	Intel Core i3	Intel Core i5 o superior
RAM	4 GB	8 GB o más
Disco duro	500 MB libres	2 GB libres
Pantalla	1366x768	1920x1080

Notas:

- El procesamiento de archivos grandes (>50 MB) requiere más RAM
- Excel COM consume recursos adicionales durante la ejecución

7. EJECUCIÓN PASO A PASO

Proceso 1: Actualización del Inventario General

Preparación (realizar una vez):

1. Instalar Python:

- Descargar desde <https://www.python.org/downloads/>
- Ejecutar instalador
- ☒ Marcar "Add Python to PATH"

- Click en "Install Now"

2. Instalar librerías:

- Abrir CMD o PowerShell
- Ejecutar: `pip install pandas openpyxl msoffcrypto-tool pywin32 unidecode`
- Esperar a que termine la instalación

3. Verificar instalación:

- Ejecutar: `python --version`
- Debe mostrar: Python 3.x.x

Ejecución diaria:

PASO 1: Descargar archivos del ERP

- Ingresar al sistema ERP de la empresa
- Descargar los siguientes reportes con fecha actual:
 - [] INVENTARIO GENERAL ACTUALIZADO
 - [] VALORIZADO GENERAL
 - [] VALORIZADO FALTANTES IMPO
 - [] VALORIZADO FALTANTES
 - [] VALORIZADO TOBERIN
- Guardar todos los archivos en la carpeta `MACRO_INV_GENERAL`
- **Importante:** No cambiar los nombres de los archivos

PASO 2: Verificar archivos maestros

- Confirmar que existen en la carpeta:
 - [] \$2025 INVENTARIO GENERAL.xlsx (plantilla)
 - [] MARCAS.xlsx
 - [] DISTRIBUCION DE MATRICES.xlsx

- [] \$2025 MATRIZ USD.xlsx
- [] 2025 INVENTARIO MYR EXISTENCIA.xlsx

PASO 3: Ejecutar la actualización

- Ubicar el archivo: actualizar_inventario_general.bat
- Hacer **doble clic** en el archivo
- Se abrirá una ventana negra (consola)

PASO 4: Monitorear el proceso

- La consola mostrará mensajes como:

```
[11:48:10] Tabla dinámica actualizada
[11:48:10] 2 tabla(s) dinámica(s) actualizada(s) exitosamente
[11:48:10] Estableciendo zoom al 80% en todas las hojas...
[11:48:10] Centrando columna EXISTENCIA...
[11:48:10] Columna encontrada: 'EXISTENCIA OCT 22' (índice 7)
[11:48:10] Activando hoja INVENTARIO como hoja predeterminada...
[11:48:10] Guardando cambios con zoom aplicado y hoja INVENTARIO activa...
[11:48:11] ☒ Cambios guardados exitosamente
[11:48:11] == Proceso completado exitosamente ==
Presione una tecla para continuar . . .
```

PASO 5: Verificar el resultado

- Al finalizar, buscar en la carpeta el archivo generado:
 - Nombre: \$2025 INVENTARIO GENERAL ACTUALIZADO
[FECHA]_[HORA].xlsx
 - Ejemplo: \$2025 INVENTARIO GENERAL ACTUALIZADO
20251022_1015.xlsx

- Abrir el archivo en Excel
- Verificar:
 - [] Columna "EXISTENCIA [MES] [DÍA]" tiene valores
 - [] Columna "COSTO PROMEDIO" tiene valores
 - [] Columna "NOMBRE ODOO" tiene datos
 - [] No hay errores #N/D o #REF

PASO 6: Presionar Enter para cerrar

- En la consola, aparecerá: Presione una tecla para continuar...
- Presionar **Enter** o cualquier tecla
- La ventana se cerrará

Tiempo estimado: 3-5 minutos (dependiendo del tamaño de los archivos)

Proceso 2: Comparación de Inventarios

¿Cuándo usar esta herramienta?

- Para validar que el proceso automatizado funciona correctamente
- Cuando se detectan discrepancias en los datos
- Para auditorías periódicas

PASO 1: Preparar archivos a comparar

- Tener disponibles:
 - Archivo inventario MANUAL (actualizado)
 - Archivo inventario AUTOMATIZADO (generado por el script)

PASO 2: Ejecutar herramienta de comparación

- Ubicar el archivo: `comparar_inv_general.bat`

- Hacer **doble clic**
- Se abrirá ventana de consola

PASO 3: Seleccionar inventario MANUAL

- Aparecerá diálogo de Windows: "Selecciona el INVENTARIO MANUAL"
- Navegar a la ubicación del archivo manual
- Seleccionar el archivo
- Click en "Abrir"

PASO 4: Seleccionar inventario AUTOMATIZADO

- Aparecerá segundo diálogo: "Selecciona el INVENTARIO ACTUALIZADO (código)"
- Seleccionar el archivo generado por el script
- Click en "Abrir"

PASO 5: Esperar el análisis

- La consola mostrará:

[10:30:15] Leyendo MANUAL: INVENTARIO MANUAL 16 OCT.xlsx

[10:30:16] Leyendo AUTO : \$2025 INVENTARIO GENERAL ACTUALIZADO
20251016_0900.xlsx

[10:30:17] Comparando 12,234 referencias comunes en 15 columnas...

[10:30:18] Columnas: ['NOMBRE ODOO', 'EXISTENCIA OCT 16', 'COSTO
PROMEDIO', ...]

[10:30:20] ☒ Listo.

[11:48:57] [INFO] Agregando comparación de EXISTENCIA: 'EXISTENCIA OCT 22'
vs 'EXISTENCIA OCT 16'

[11:48:57] Comparando 13923 referencias comunes en 19 columnas...

[11:48:57] Columnas: ['CLASIFICACION', 'COSTO PROMEDIO', 'Dif linea', 'Dif
marca', 'Dif sub-linea', 'INV BODEGA GERENCIA', 'LIDER LINEA', 'LINEA COPIA',
'Linea sistema', 'MARCA copia', 'Marca sistema', 'NOMBRE LISTA', 'NOMBRE

MYR', 'NOMBRE ODOO', 'REFERENCIA', 'SUB-LINEA COPIA', 'Sub- linea sistema', 'TOTAL INV', 'EXISTENCIA OCT 22']

[11:48:58] Escribiendo resultado:

C:\Users\jperez\Desktop\Proyectos\MACRO_INV_GENERAL\DIFERENCIAS INVENTARIO 20251022_1148.xlsx

[11:48:58] ☒ Listo.

Archivo generado:

C:\Users\jperez\Desktop\Proyectos\MACRO_INV_GENERAL\DIFERENCIAS INVENTARIO 20251022_1148.xlsx

PASO 6: Revisar el reporte de diferencias

- Abrir el archivo generado: DIFERENCIAS INVENTARIO [FECHA]_[HORA].xlsx
- El archivo contiene una tabla con 4 columnas:
 - **REFERENCIA:** Código del producto
 - **COLUMNA:** Nombre del campo que difiere
 - **VALOR_MANUAL:** Valor en el inventario manual
 - **VALOR_AUTO:** Valor en el inventario automatizado
- Ejemplo:

REFERENCIA	COLUMNA	VALOR_MANUAL	VALOR_AUTO
95276	EXISTENCIA OCT 16	150	145
500845	COSTO PROMEDIO	12500	12480
FP-1234	NOMBRE ODOO	PRODUCTO X	PRODUCTO X (NUEVO)

PASO 7: Analizar diferencias

- Si hay diferencias: revisar:
 - [] ¿Los archivos valorizados son del mismo día/hora?
 - [] ¿La plantilla está actualizada?
 - [] ¿Hubo cambios manuales en alguno de los archivos?

Tiempo estimado: 2-3 minutos

Proceso 3: Actualización Vespertina (Macro Inv Tarde)

¿Cuándo ejecutar?

- Al final del día, después de las 4:00 PM

Diferencias con el proceso matutino:

- Misma funcionalidad
- Carpeta diferente: MACRO_INV_GENERAL/Macro Inv Tarde/
- Archivos independientes

Pasos:

1. Descargar archivos actualizados del ERP
2. Guardar en carpeta: Macro Inv Tarde
3. Ejecutar: actualizar_inventario_general_tarde.bat
4. Seguir mismos pasos que en Proceso 1

Manejo de Errores Comunes

Error: "No se encontró Python"

- **Causa:** Python no está instalado o no está en el PATH
- **Solución:**
 - a. Abrir CMD
 - b. Ejecutar: `python --version`
 - c. Si falla, reinstalar Python marcando "Add to PATH"

Error: "No module named 'pandas'"

- **Causa:** Librerías no instaladas
- **Solución:**

```
pip install pandas openpyxl msoffcrypto-tool pywin32 unidecode
```

Error: "No se encontró el archivo [NOMBRE]"

- **Causa:** Archivo requerido no está en la carpeta
- **Solución:**
 - a. Verificar que el archivo exista
 - b. Confirmar que el nombre empiece con el prefijo esperado
 - c. Verificar extensión (.xlsx, .xlsm)

Error: "Contraseña incorrecta"

- **Causa:** Archivo cifrado con contraseña diferente
- **Solución:**
 - a. Editar el script Python
 - b. Buscar: `PASSWORDS_TRY = ["Compras2025", "Compras2026"]`
 - c. Agregar la contraseña correcta a la lista

Error: "Excel COM no disponible"

- **Causa:** Excel no está instalado o pywin32 falta
- **Solución:**
 - a. Instalar Microsoft Excel
 - b. Ejecutar: `pip install pywin32`
 - c. Ejecutar: `python -m pywin32_postinstall -install`

Error: "Memory Error" o proceso muy lento

- **Causa:** Archivos muy grandes o poca RAM
- **Solución:**
 - a. Cerrar otros programas
 - b. Aumentar RAM del equipo (si es posible)
 - c. Procesar archivos en partes más pequeñas

7. EJEMPLOS Y EVIDENCIAS

Antes de la automatización:

Proceso manual (3-4 horas):

1. Abrir 8 archivos Excel diferentes
2. Copiar y pegar datos manualmente
3. Aplicar fórmulas en Excel:

```
=BUSCARV(A2, 'VALORIZADO GENERAL'!A:Z, 15, 0)
```

4. Revisar y corregir errores #N/D
5. Actualizar clasificaciones una por una
6. Verificar coherencia de datos

7. Guardar y proteger archivo

Problemas frecuentes:

- Errores de tipeo al copiar/pegar
- Fórmulas rotas por cambios en estructura
- Tiempo excesivo invertido
- Alta probabilidad de error humano

Después de la automatización:

Proceso automatizado (3-5 minutos):

1. Descargar archivos del ERP
2. Doble clic en archivo .bat
3. Esperar 3 minutos
4. Verificar archivo generado

Beneficios:

- ☒ 98% reducción en tiempo de proceso
- ☒ Eliminación de errores de tipeo
- ☒ Consistencia en normalización de referencias
- ☒ Proceso replicable y auditable
- ☒ Logs detallados de cada ejecución

Captura de consola durante ejecución:

[11:50:34] == Inicio actualización de inventario ==

[11:50:34] Cargando datos externos...

[11:50:34] Abriendo inventario actualizado (ERP): INVENTARIO GENERAL
ACTUALIZADO 16 DE OCTUBRE DE 2025 (MAÑANA).xls

[11:50:36] Abrir: VALORIZADO GENERAL 16 OCT.xlsx
[11:50:36] Abrir: VALORIZADO FALTANTES IMPO 16 OCT.xlsx
[11:50:36] Abrir: VALORIZADO FALTANTES 16 OCT.xlsx
[11:50:36] Abrir: VALORIZADO TOBERIN 16 OCT.xlsx
[11:50:36] Abriendo Matriz USD: \$2025 MATRIZ USD.xlsx
[11:50:40] Matriz USD: 13824 referencias disponibles para actualizar NOMBRE LISTA
[11:50:40] Matriz USD: 13299 referencias de lista de precios disponibles
[11:50:40] Abriendo archivo MARCAS: MARCAS.xlsx

Ejemplo de log de comparación:

Usando py launcher...
[11:51:17] Leyendo MANUAL:
C:\Users\jperez\Desktop\Proyectos\MACRO_INV_GENERAL\2025 INVENTARIO GENERAL.xlsx
[11:51:17] Intento openpyxl: 2025 INVENTARIO GENERAL.xlsx
[11:51:17] openpyxl falló: File is not a zip file
[11:51:17] Intento desenscriptar con clave 'Compras2025'...
[11:51:17] Clave no válida: The file could not be decrypted with this password
[11:51:17] Intento desenscriptar con clave 'Compras2026'...
[11:51:18] Desenscriptado OK

Ejemplo de archivo generado (DIFERENCIAS):

El archivo Excel contiene:

Hoja: Diferencias

REFERENCIA	COLUMNA	VALOR_MANUAL	VALOR_AUTO
95276	EXISTENCIA OCT 16	150	145
500845	COSTO PROMEDIO	12500	12480
FP-1234	NOMBRE ODOO	PRODUCTO X	PRODUCTO X (ACTUALIZADO)
789012	Marca sistema	MARCA A	MARCA B
345678	EXISTENCIA OCT 16	200	198

Interpretación:

- **Fila 1:** La referencia 95276 tiene 5 unidades menos en el automatizado (posible venta posterior)
- **Fila 2:** Diferencia mínima en costo (redondeo o actualización)
- **Fila 3:** Nombre actualizado en el sistema
- **Fila 4:** Marca corregida por reglas de negocio
- **Fila 5:** Diferencia menor en existencia

8. HISTORIAL DE CAMBIOS

Fecha	Versión	Descripción del cambio	Responsable
15/10/2025	1.0	Creación inicial del script de actualización	Jefferson Pérez

9. RECOMENDACIONES FINALES

Buenas prácticas de uso:

1. Respaldo regular:

- Mantener copia del archivo plantilla original
- Guardar versiones anteriores del inventario generado
- Crear carpeta "RespalDOS" con archivos históricos

2. Validación periódica:

- Ejecutar comparación cada semana
- Revisar logs en busca de errores
- Documentar diferencias significativas

3. Actualización de archivos maestros:

- MARCAS.xlsx: actualizar cuando se agreguen nuevas marcas propias
- DISTRIBUCION DE MATRICES.xlsx: actualizar cuando cambien gestores o clasificaciones
- \$2025 MATRIZ USD.xlsx: actualizar con nuevos productos

4. Nomenclatura de archivos:

- No renombrar archivos descargados del ERP

- Mantener prefijos originales
- El script busca por prefijo, no por nombre exacto

5. Monitoreo de errores:

- Leer los logs completamente antes de cerrar la consola
- Si aparece "WARNING" o "ERROR", investigar causa
- Guardar copia de pantalla de errores para soporte

Seguridad:

1. Contraseñas:

- Las contraseñas están codificadas en el script
- No compartir el código fuente con personal no autorizado
- Actualizar contraseñas en el código si cambian en el sistema

2. Acceso a archivos:

- Solo personal autorizado debe ejecutar los scripts
- Archivos generados contienen información sensible
- No compartir por correo sin cifrado

3. Archivos temporales:

- El script limpia automáticamente archivos temporales
- Si quedan residuos en C:\Users\$\$Usuario\AppData\Local\Temp, eliminarlos manualmente

Mantenimiento:

1. Actualización anual:

- Al cambiar de año, actualizar:
 - FN_INV_PLANTILLA = "\$2026 INVENTARIO GENERAL.xlsx"
 - PFX_MATRIZ_USD = "\$2026 MATRIZ USD"

- `SHEET_MATRIZ_2025 = "2026"`
- Verificar que estructura de archivos ERP no haya cambiado

2. Optimización:

- Si el proceso toma más de 10 minutos, considerar:
 - Aumentar RAM del equipo
 - Dividir procesamiento en lotes
 - Actualizar versión de Python y librerías

3. Documentación:

- Mantener este documento actualizado
- Registrar cambios en la sección "Historial de cambios"
- Documentar problemas recurrentes y soluciones

Soporte técnico:

Para problemas con Python/Librerías:

- Documentación pandas: <https://pandas.pydata.org/docs/>
- Documentación openpyxl: <https://openpyxl.readthedocs.io/>
- Stack Overflow: <https://stackoverflow.com/> (buscar errores específicos)

Para problemas con Excel COM:

- Verificar que Excel esté correctamente instalado
- Ejecutar: `python -m pywin32_postinstall -install` después de instalar pywin32
- Reiniciar equipo después de instalar pywin32

Para problemas con archivos del ERP:

- Contactar al área de sistemas
- Verificar permisos de descarga
- Confirmar formato y estructura de reportes

10. GLOSARIO DE TÉRMINOS

Término	Definición
ERP	Enterprise Resource Planning - Sistema de gestión empresarial integrado
COM	Component Object Model - Tecnología de Microsoft para comunicación entre aplicaciones
DataFrame	Estructura de datos tabular de pandas (similar a una tabla Excel)
Normalización	Proceso de estandarizar formatos de datos para comparación
Path	Ruta de archivo o carpeta en el sistema operativo
Referencia	Código único que identifica un producto en el inventario
Valorizado	Reporte que incluye cantidades y costos de productos
Existencia	Cantidad disponible de un producto en inventario
Costo promedio	Precio promedio ponderado de compra de un producto
Marca propia	Marca que pertenece directamente a la empresa
Línea	Categoría principal de productos
Sub-línea	Subcategoría dentro de una línea
Gestor/Líder	Persona responsable de una línea de productos
Clasificación	Categorización de productos según criterios de negocio
Faltantes	Productos pendientes de recibir o completar
Toberín	Ubicación física de almacenamiento

Remisión en consignación	Productos entregados pero no facturados
Matriz USD	Tabla maestra de precios en dólares
.bat (Batch)	Archivo ejecutable de comandos de Windows
Timestamp	Marca de fecha y hora
Header/Encabezado	Primera fila de una tabla que contiene nombres de columnas
BytesIO	Objeto de Python que simula un archivo en memoria
**Cifrado/Enc	